

Report

Assignment 01: Multilingual Tokenization and Language Modeling

Machine Learning for Statistical NLP: Advanced

Anonymous ACL submission

1 Part 1: The Tokenizers

I started the assignment by setting up a virtual environment on the server and installing all the necessary packages (such as jupyter and pytorch, etc.). I then registered the virtual environment as a jupyter kernel and connected the server to VS Code so I had a clean, convenient working environment. I first checked that all the language files existed and the contents of README.txt and metadata.json. Based on the data, I wrote the following predictions for the checkpoint:

Which language do you expect to require the most tokens? For the character tokenizer, each language should have roughly similar token counts (since the characters are balanced in the data). For BPE, I expect Chinese, because it's a character-based language, and one character can already be a word. So two different characters might not occur with each other that much, therefore there might be more single character tokens. It might also not be as morphologically rich as Turkish, or even English in comparison.

How do you expect the larger BPE vocabulary to change the representation of each language? I think that each language will have less tokens in total and per sentence and the number of characters per token will go up (since the larger vocabulary will allow longer sequences to become tokens). I expect that Turkish especially will receive noticeably less tokens since it might originally have had more tokens because it's morphologically rich. I expect a similar outcome for English, but less so. For Chinese, the number of characters per token might be lower than for English and Turkish due to it being a character-based language. But it might lead to tokens being expressions or even short utterances.

What kinds of units do you expect BPE to learn in English, Turkish, and Chinese? For English, I mostly expect BPE to learn tokens that

are words, and maybe some suffixes. For Turkish - a morphologically richer language -, I expect it to learn more affixes and stems, as well as words. For Chinese, I expect it to mostly learn characters, possibly words made up of several characters or even expressions made up of several characters.

Which tokenizer do you expect to work best for language modeling? I think the small BPE tokenizer might be the sweet spot. It will probably create larger meaningful chunks while not putting rare larger tokens at a disadvantage in training by creating too rare tokens.

Then I built the character-level tokenizer by first creating a set of all the individual characters in the training data (9442 in total). I turned the set into a sorted list and added <unk> at index 0 to handle unknown characters, making the total number of characters 9443. After that, I created the character-to-integer and integer-to-character mappings using two dictionaries. I wrote the encoder and decoder functions with <unk> as a fallback and then saved the 9443-item vocabulary for the character-level tokenizer.

For the BPE tokenizers, I first tried to implement the HuggingFace BPE model. I chose the Whitespace pre-tokenizer in order to split punctuation from any words to prevent BPE from learning punctuation as separate tokens that can be merged. However, the HuggingFace BPE tokenizer did not work well for my purposes. I realized that using their *vocab_size* parameter and setting it to 2000 for the small vocabulary BPE tokenizer meant that I had to set *limit_alphabet* to 1000. This means that the majority of the original characters would never even be included in the model. I assumed this would affect Chinese with its many characters disproportionately. Therefore, I decided to switch to the SentencePiece BPE tokenizer implementation.

I chose a vocabulary size of 3000 for the smaller BPE tokenizer and a vocabulary size of 10 000 for the larger one. I originally tried training the smaller

tokenizer with a vocabulary size of 2k. This did not work, however, if I wanted to keep the coverage (SentencePiece implementation) higher. I first tried using a coverage of 0.9995, which is recommended for languages like Chinese. But this required a minimum of 6394 vocabulary items. In order to keep the coverage as high as possible, I decided to increase the vocabulary size by 1000 (and decreased the coverage to 0.9929). 10k vocabulary items then seemed like a reasonable step up from 3k, which worked well with the same coverage. The <unk> ID was again put at index 0.

I trained both BPE tokenizers and then evaluated them on the validation data. I wrote an evaluation function to read each language file and count the number of sentences, characters and tokens as well as the number of unknown characters. This was also used to calculate the average tokens per sentence and the average characters per token. I then used a function to encode each validation language file with each tokenizer and get the results from the evaluation function. These results were presented in a table. Lastly, I looked at three sentences from each language in all three tokenizer conditions.

The character-level tokenizer, predictably, produces the highest number of total tokens for each language. Both BPE tokenizers produce lower numbers of tokens across all three languages compared to that. The 3k BPE tokenizer led to higher numbers of tokens, especially for Chinese (358 234), compared to the 10k BPE tokenizer. Still, Chinese has the highest number of tokens with the 10k tokenizer, too (285 630).

With the character-level tokenizer, each language has the highest number of average tokens per sentence, respectively. For English and Turkish, this number decreases with the 3K BPE tokenizer, and even more so with the 10k one. English always has less tokens per sentence on average than Turkish. For Chinese, that number barely decreases between the character-level (38.55) and the 3k BPE tokenizer (37.55). There is only a significant difference with the 10k BPE tokenizer (29.90). The average character per token increases slightly from 1.0 from the character-level tokenizer to the 3k one - but barely so for Chinese (1.03). For Turkish (3.27) and English (3.24), it increases noticeably with the 10k tokenizer, but again barely any increase for Chinese (1.29). With the character-level tokenizer, English (49) has more unknown tokens than Turkish (15). For both BPE encoders, English has 90 unknown tokens, and Turkish 102. Chinese has by far

the highest number of unknown tokens with every tokenizer, especially the BPE ones. The increase between the character-level tokenizer and the BPE ones is dramatic: from 258 to 6971 unknown tokens for both BPE tokenizers.

For all results see table 1.

After looking at a few examples from each language, here are my insights: For English, the 3k tokenizer split words into smaller pieces like "play" to "p | l | l | a | y | e | d", while the 10k one produced larger pieces like "played" and "two". This matches the increase in average characters per token. "ed" in "played" happens to be a meaningful suffix in English, but not all pieces the tokenizers learned were meaningful.

It is similar for Turkish, with "okullarda" being fragmented into "o | l | k | l | u | l | l | a | r | d | a" with the 3k tokenizer and into "okul | l | a | r | d | a" with the 10k one. "lar" and "da" constitute the plural suffix and a locative suffix, respectively. They were both recognized as tokens in the 3k model. For the 10k model, however, "larda" ended up merged. Contrary to the 3k results, "okul" (meaning "school") was also merged with the 10k tokenizer. Therefore both BPE tokenizers were capable of merging actual linguistic units - but in different ways and seemingly not reliably. It is interesting that the correctly merged "lar" and "da" ended up together as one token in the 10k model.

For Chinese, the 3k tokenizer mostly learned single character units. With the 10k tokenizer, there are more multi-character units like 以上, which is a lexical unit made out of two components, and 大学, whose components have individual meanings but form a different word together. Many of them seem to consist of only two characters though. This matches with the relatively low chars/token number for Chinese, compared to English and Turkish. In the two tokens observed more closely, 10k tokenizer has managed to merge characters that actually occur together in this way in natural language.

The differences in results for each language can probably be explained by their structures. Chinese inherently has more characters than Turkish or English, therefore with the limited vocabulary space in the 3k tokenizer Chinese has to receive a big part of it, leaving less to perform multi-character merges. With the 10k vocabulary, there is more room for character merges for Chinese.

Chinese's character-based structure is likely also the reason for it having so many more unknown characters with the BPE tokenizers. There are a lot

Tokenizer	Lang.	Vocab.	Sent.	Chars.	Tokens	Tok./Sent.	Chars/Tok.	UNK
Character	en	9443	4540	368269	368269	81.12	1.00	49
Character	tr	9443	3735	368383	368383	98.63	1.00	15
Character	zh	9443	9554	368266	368266	38.55	1.00	258
BPE-3k	en	3000	4540	368269	226896	49.98	1.62	90
BPE-3k	tr	3000	3735	368383	245266	65.67	1.50	102
BPE-3k	zh	3000	9554	368266	358234	37.50	1.03	6971
BPE-10k	en	10000	4540	368269	113850	25.08	3.23	90
BPE-10k	tr	10000	3735	368383	112661	30.16	3.27	102
BPE-10k	zh	10000	9554	368266	285630	29.90	1.29	6971

Table 1: Tokenization statistics for character-level and BPE tokenizers across English (en), Turkish (tr), and Chinese (zh). “Vocab.” denotes vocabulary size, “Sent.” the number of sentences, “Tok./Sent.” the average number of tokens per sentence, “Chars/Tok.” the average number of characters per token, and “UNK” the total number of unknown tokens.

of distinct characters, and with the lowered character coverage, rare Chinese characters are not receiving their own vocabulary entries. This might be a limitation of my approach.

An interesting result to investigate further is that for the 3k BPE tokenizer, the tokenization of Chinese remained almost at character-level (1.03 chars/token). While the average number of characters per token increased with the 10k tokenizer (1.29), it stayed noticeably behind the averages for English and Turkish (both roughly 3.25). Even with the 3k BPE tokenizer, English and Turkish had higher char/token numbers (1.63 and 1.50, respectively). Chinese therefore needs more tokens to encode the same number of text. It will thus be interesting to see if these differences in tokenization between the languages will also be reflected in the performance of the language models.

2 Part 2: The Language Models

In Part 2, I first reloaded the saved character-level tokenizer vocabulary and rebuilt the text encoder from part 1, including the <unk> character. I also reloaded the saved BPE tokenizers, as well as recreated the encoding wrappers from part 1. Then I wrote a function to create token streams (one continuous list with all the characters for each data file), which also adds *eos* after every sentence, so that the language model can later learn sentence boundaries.

I copied the tokenizer configurations from part 1 and then built the training streams for every tokenizer from `balanced.txt`. I used `balanced.txt` because it contains all the training data already concatenated in mixed order. After that, I built the validation streams, using the single language files from the validation directory and concatenating them into a single stream before tokenizing with

each tokenizer.

I built a custom dataset to dive each token stream into token windows with a context length of 128. For each window, the targets for next-token prediction consisted of the same window shifted by one token. I then used the `pytorch Dataloader` to group the windows into batches of 64 for the training datasets for efficient parallel GPU training. The batches were also shuffled between each epoch. This resulted in input and target tensors of size [64, 128] (64 sequences of 128 tokens each). This process was repeated for the validation data minus the shuffling.

Finally, I built a decoder-only Transformer language model using. It is trained from scratch with causal self-attention. It has 2 Transformer layers with 4 attention heads each. The hidden/embedding dimension is 256, the feed-forward dimension is 1024, the context length 128 and dropout is 0.1. The vocabulary size depends on the tokenizer used beforehand. The model turns each token ID into a 256-dimensional vector, it does the same for each position within the context length (learned token and positional embeddings). The causal attention mask prevents the model from looking at future tokens. In the linear output layer, the 256-dimensional hidden representations are converted into vocabulary scores. The architecture of the Transformer model remained the same over all three tokenizer conditions. But as mentioned, the vocabulary sizes differed and therefore the input embedding and output layer sizes differed too.

The parameter counts of the models were computed afterwards and can be seen in table 2. A larger vocabulary increases the parameters, i.e. the size of the model. It is worth noting that the vocabulary sizes increased by 1 after the addition of the *eos* ID compared to part 1.

Model	Vocab.	Total	Embedding	Output
Character	9,444	6,457,060	2,417,664	2,427,108
BPE-3k	3,001	3,151,801	768,256	771,257
BPE-10k	10,001	6,742,801	2,560,256	2,570,257

Table 2: Number of parameters for each model configuration. “Vocab.” denotes vocabulary size, while “Embedding ” and “Output” denote the number of parameters in the embedding and output layers, respectively.

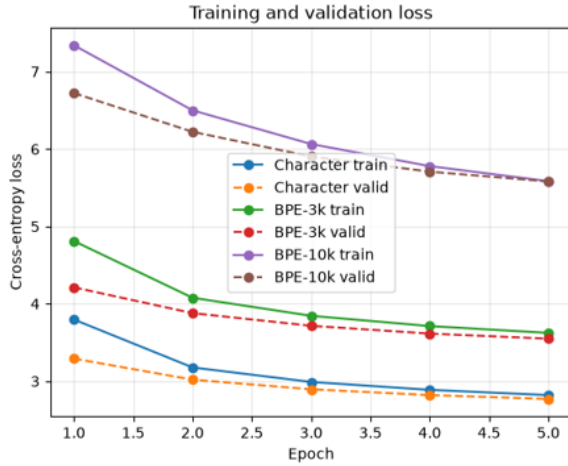


Figure 1: Training and validation loss.

Three models were trained, one for each tokenizer, using a learning rate of $3e-4$ and AdamW optimizer. The models were trained for next-token prediction using cross-entropy loss. Each model was saved after training. The loss for training and validation data was calculated, using one helper function each, recorded and then plotted after training had finished (see figure 1).

As a training-control policy I chose number of complete passes through the corpus (epochs). An advantage of this is that it exposes the models to the same amount of underlying text. However, the tokenizers produced different amounts of tokens, therefore the computational cost per epoch differs between the models. Each model was trained for 5 epochs.

In part 2, I observed the following: The parameter counts show that a larger vocabulary increases the model parameters. For training, the character-level tokenizer needed 8,980,877 tokens to represent the training text. The BPE 3k tokenizer needed 6,780,586 and the BPE 10k tokenizer needed 4,221,411 tokens. The parameters counts for the character model were 6,457,060. They were 3,151,801 for the 3k BPE model and 6,742,801 for the 10k BPE model.

This is because the model produces an embedding for every possible token and an output score for every possible token. Naturally, the BPE 10k model has the most model parameters. This is interesting because the 10k tokenizer allowed more merges and therefore produced less total tokens than the other two tokenizers. So the 10k language model has the least training tokens but is the largest model. The 3k BPE tokenizer has more training tokens than the 10k one (but less than the character-level tokenizer), but it is the smallest model - less than half the size of the 10k model. This could mean that while a larger BPE vocabulary allows longer sequences and therefore fewer tokens in total, there is a trade-off: the model needs a bigger vocabulary layer to be able to produce a 256-dimensional embedding for every item in the vocabulary and to produce a prediction score for every vocabulary item during next-token prediction.

3 Part 3: Evaluation and Analysis

WHAT I DID

WHY I DID IT

HOW TO REPRODUCE

WHAT I LEARNED

4 AI Log

To produce a PDF file, pdfL^AT_EX is strongly recommended (over original L^AT_EX plus dvips+ps2pdf or dvipdf). The style file acl.sty can also be used with luaL^AT_EX and XeL^AT_EX, which are especially suitable for text in non-Latin scripts. The file acl_lualatex.tex in this repository provides an example of how to use acl.sty with either luaL^AT_EX or XeL^AT_EX.

5 Preamble

The first line of the file must be

```
\documentclass[11pt]{article}
```

To load the style file in the review version:

```
\usepackage[review]{acl}
```

For the final version, omit the review option:

```
\usepackage{acl}
```

To use Times Roman, put the following in the preamble:

```
\usepackage{times}
```

Command	Output	Command	Output
<code>\"a</code>	ä	<code>\c c</code>	ç
<code>\^e</code>	ê	<code>\u g</code>	ğ
<code>\`i</code>	ì	<code>\l</code>	ł
<code>\.I</code>	İ	<code>\~n</code>	ñ
<code>\o</code>	ø	<code>\H o</code>	õ
<code>\'u</code>	ú	<code>\v r</code>	ř
<code>\aa</code>	å	<code>\ss</code>	ß

Table 3: Example commands for accented characters, to be used in, *e.g.*, BibTeX entries.

(Alternatives like txfonts or newtx are also acceptable.)

Please see the L^AT_EX source of this document for comments on other packages that may be useful.

Set the title and author using `\title` and `\author`. Within the author list, format multiple authors using `\and` and `\And` and `\AND`; please see the L^AT_EX source for examples.

By default, the box containing the title and author names is set to the minimum of 5 cm. If you need more space, include the following in the preamble:

```
\setlength\titlebox{<dim>}
```

where `<dim>` is replaced with a length. Do not set this length smaller than 5 cm.

6 Document Body

6.1 Footnotes

Footnotes are inserted with the `\footnote` command.¹

6.2 Tables and figures

See Table 3 for an example of a table and its caption. **Do not override the default caption sizes.**

As much as possible, fonts in figures should conform to the document fonts. See Figure 2 for an example of a figure and its caption.

Using the `graphicx` package `graphics` files can be included within figure environment at an appropriate point within the text. The `graphicx` package supports various optional arguments to control the appearance of the figure. You must include it explicitly in the L^AT_EX preamble (after the `\documentclass` declaration and before `\begin{document}`) using `\usepackage{graphicx}`.

¹This is a footnote.



Figure 2: A figure with a caption that runs for more than one line. Example image is usually available through the `mwe` package without even mentioning it in the preamble.

6.3 Hyperlinks

Users of older versions of L^AT_EX may encounter the following error during compilation:

```
\pdfendlink ended up in different nest-
ing level than \pdfstartlink.
```

This happens when pdfL^AT_EX is used and a citation splits across a page boundary. The best way to fix this is to upgrade L^AT_EX to 2018-12-01 or later.

6.4 Citations

Table 4 shows the syntax supported by the style files. We encourage you to use the `natbib` styles. You can use the command `\citet` (cite in text) to get “author (year)” citations, like this citation to a paper by ?. You can use the command `\citep` (cite in parentheses) to get “(author, year)” citations (?). You can use the command `\citealp` (alternative cite without parentheses) to get “author, year” citations, which is useful for using citations within parentheses (*e.g.* ?).

A possessive citation can be made with the command `\citepos`. This is not a standard `natbib` command, so it is generally not compatible with other style files.

6.5 References

The L^AT_EX and BibTeX style files provided roughly follow the American Psychological Association format. If your own bib file is named `custom.bib`, then placing the following before any appendices in your L^AT_EX file will generate the references section for you:

```
\bibliography{custom}
```

You can obtain the complete ACL Anthology as a BibTeX file from <https://aclweb.org/>

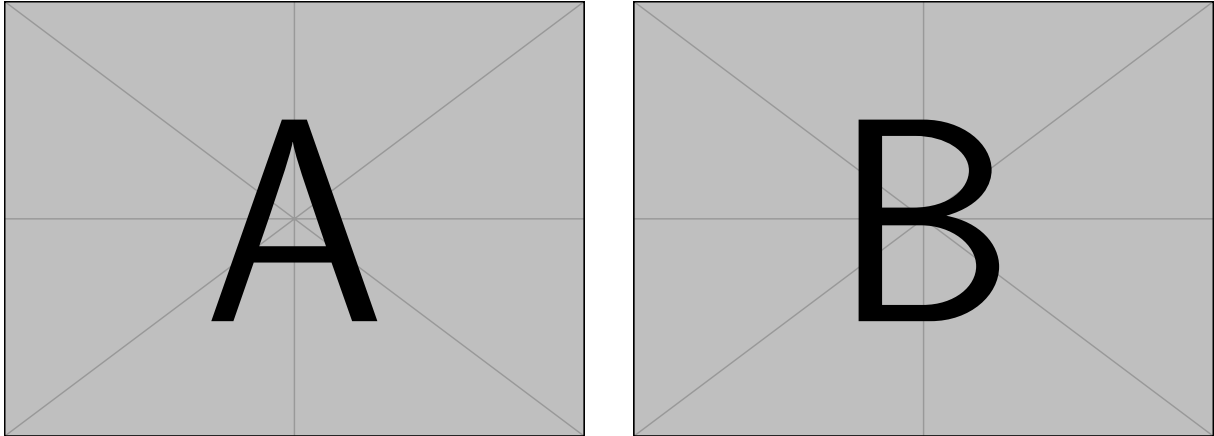


Figure 3: A minimal working example to demonstrate how to place two images side-by-side.

Output	natbib command	ACL only command
(?)	<code>\citep</code>	
?	<code>\citealp</code>	
?	<code>\citet</code>	
(?)	<code>\citeyearpar</code>	
?’s (?)		<code>\citeposs</code>

Table 4: Citation commands supported by the style file. The style is based on the natbib package and supports all natbib citation commands. It also supports commands defined in previous ACL style files for compatibility.

[anthology/anthology.bib.gz](#). To include both the Anthology and your own .bib file, use the following instead of the above.

```
\bibliography{anthology,custom}
```

Please see Section 7 for information on preparing BibT_EX files.

6.6 Equations

An example equation is shown below:

$$A = \pi r^2 \tag{1}$$

Labels for equation numbers, sections, subsections, figures and tables are all defined with the `\label{label}` command and cross references to them are made with the `\ref{label}` command.

This an example cross-reference to Equation 1.

6.7 Appendices

Use `\appendix` before any appendix section to switch the section numbering over to letters. See Appendix ?? for an example.

7 BibT_EX Files

Unicode cannot be used in BibT_EX entries, and some ways of typing special characters can disrupt

BibT_EX’s alphabetization. The recommended way of typing special characters is shown in Table 3.

Please ensure that BibT_EX records contain DOIs or URLs when possible, and for all the ACL materials that you reference. Use the `doi` field for DOIs and the `url` field for URLs. If a BibT_EX entry has a URL or DOI field, the paper title in the references section will appear as a hyperlink to the paper, using the `hyperref` L^AT_EX package.

Limitations